TASK 14

1.Source code

Product Class

Product.java

```
import java.io.Serializable;

/*
 * Product class represents an inventory item
 */
public class Product implements Serializable {
   private static final long serialVersionUID = 1L;
   private int id;
   private String name;
   private int quantity;
   private double price;
   public Product(int id, String name, int quantity, double price) {
      this.id = id;
      this.name = name;
      this.quantity = quantity;
      this.price = price;
   }
   public int getId() {
      return id;
   }
   public void setQuantity(int quantity) {
      this.quantity = quantity;
```

```
    }
    public void setPrice(double price) {
        this.price = price;
    }
    @Override
    public String toString() {
        return "ID: " + id +
                " | Name: " + name +
                " | Quantity: " + quantity +
                " | Price: " + price;
    }
}
```

Inventory Management Application

InventoryApp.java

```
import java.io.*;
import java.util.*;

/*
 * Task 14: Console-Based Inventory Management System
 */
public class InventoryApp {

    private static final String FILE_NAME = "inventory.txt";
    private static HashMap<Integer, Product> inventory = new HashMap<>();
```

```
public static void main(String[] args) {

    loadInventory(); // load data if file exists

    Scanner sc = new Scanner(System.in);

    while (true) {
        System.out.println("\n=== Inventory Management System ===");
        System.out.println("1. Add Product");
        System.out.println("2. Update Product");
        System.out.println("3. Delete Product");
        System.out.println("4. View Inventory");
        System.out.println("5. Exit");

        System.out.print("Enter choice: ");
        int choice = sc.nextInt();

        switch (choice) {
            case 1 -> addProduct(sc);
            case 2 -> updateProduct(sc);
            case 3 -> deleteProduct(sc);
            case 4 -> viewInventory();
            case 5 -> {
                saveInventory();   // 🔴 THIS CREATES inventory.txt
```

```
                System.out.println("Inventory saved. Exiting...");
                sc.close();
                return;
            }
            default -> System.out.println("Invalid choice!");
        }
    }
}

// Add product
private static void addProduct(Scanner sc) {
    System.out.print("Enter Product ID: ");
    int id = sc.nextInt();

    if (inventory.containsKey(id)) {
        System.out.println("Product ID already exists!");
        return;
    }

    sc.nextLine(); // clear buffer
    System.out.print("Enter Product Name: ");
    String name = sc.nextLine();

    System.out.print("Enter Quantity: ");
    int qty = sc.nextInt();
```

```
    System.out.print("Enter Price: ");
    double price = sc.nextDouble();

    inventory.put(id, new Product(id, name, qty, price));
    System.out.println("Product added successfully.");
}

// Update product
private static void updateProduct(Scanner sc) {
    System.out.print("Enter Product ID to update: ");
    int id = sc.nextInt();

    Product p = inventory.get(id);
    if (p == null) {
        System.out.println("Product not found.");
        return;
    }

    System.out.print("Enter new quantity: ");
    p.setQuantity(sc.nextInt());

    System.out.print("Enter new price: ");
    p.setPrice(sc.nextDouble());
```

```
    System.out.println("Product updated successfully.");
}

// Delete product
private static void deleteProduct(Scanner sc) {
    System.out.print("Enter Product ID to delete: ");
    int id = sc.nextInt();

    if (inventory.remove(id) != null) {
        System.out.println("Product deleted.");
    } else {
        System.out.println("Product not found.");
    }
}

// View inventory
private static void viewInventory() {
    if (inventory.isEmpty()) {
        System.out.println("Inventory is empty.");
        return;
    }

    System.out.println("\n--- Inventory Summary ---");
    for (Product p : inventory.values()) {
        System.out.println(p);
```

```
    }
}

// Save inventory to file (CREATES inventory.txt)
private static void saveInventory() {
    try (ObjectOutputStream oos =
                new ObjectOutputStream(new FileOutputStream(FILE_NAME))) {

        oos.writeObject(inventory);

    } catch (IOException e) {
        System.out.println("Error saving inventory: " + e.getMessage());
    }
}

// Load inventory from file
@SuppressWarnings("unchecked")
private static void loadInventory() {
    File file = new File(FILE_NAME);
    if (!file.exists()) return;

    try (ObjectInputStream ois =
                new ObjectInputStream(new FileInputStream(FILE_NAME))) {

        inventory = (HashMap<Integer, Product>) ois.readObject();
```

```
        } catch (Exception e) {
            System.out.println("Error loading inventory.");
        }
    }
}
```

2.Output:

```
=== Inventory Management System ===
1. Add Product
2. Update Product
3. Delete Product
4. View Inventory
5. Exit
Enter choice: 1
Enter Product ID: 101
Enter Product Name: laptop
Enter Quantity: 34
Enter Price: 50000
Product added successfully.

=== Inventory Management System ===
1. Add Product
2. Update Product
3. Delete Product
4. View Inventory
5. Exit
Enter choice: 4

--- Inventory Summary ---
ID: 101 | Name: laptop | Quantity: 34 | Price: 50000.0

=== Inventory Management System ===
1. Add Product
2. Update Product
3. Delete Product
4. View Inventory
5. Exit
Enter choice: 5
Inventory saved. Exiting...
```